

Теоретический конспект №5

Тема: «Deep» в Deep Reinforcement Learning

Связано с: [note 1.md](#) · [note 2.md](#) · [note 3.md](#) · [note 4.md](#)

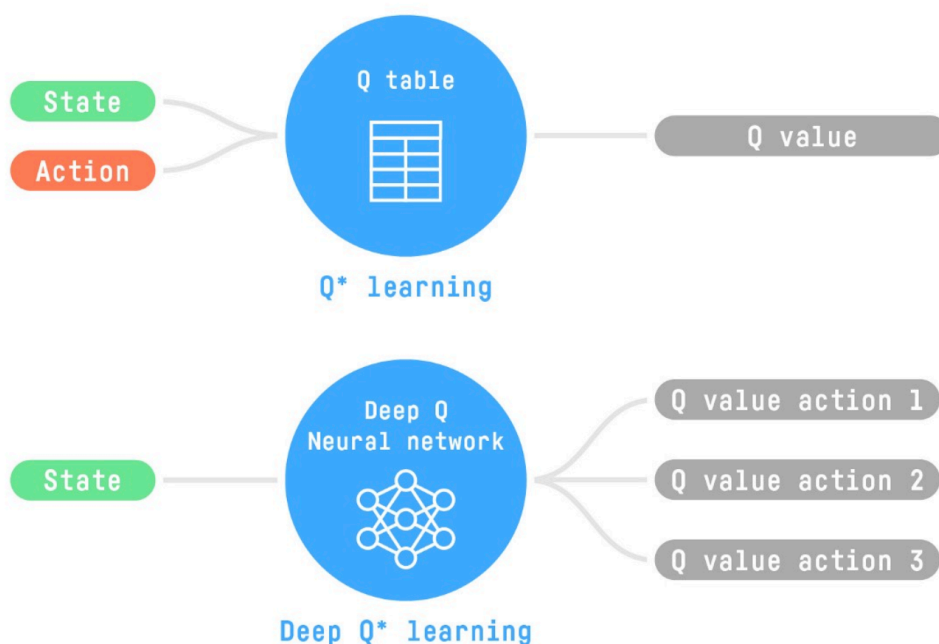
1. Напоминание: классическое Reinforcement Learning

До 2013–2014 годов большинство RL-алгоритмов использовали **табличные методы**.

Пример — Q-Learning. Он хранит таблицу $Q(s, a)$, где каждая ячейка соответствует «ценности» выполнения действия a в состоянии s .

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

Эта таблица обновляется с опытом агента.



Пример Q-таблицы

Проблема табличного подхода

Он **работает только при малом числе состояний**.

Например:

- В игре «FrozenLake» (4×4) — всего 16 состояний → ок.
- В игре «Atari Breakout» (экран 210×160×3) → $210 \times 160 \times 3 = 100,800$ пикселей! Невозможно построить таблицу на миллионы состояний.

2. Появление “Deer” — нейросетевой аппроксиматор

Здесь начинается **революция Deer RL**

Вместо таблицы $Q(s, a)$, мы используем **глубокую нейронную сеть**, которая **аппроксимирует (приближает)** функцию $Q(s, a)$:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

где:

- θ — параметры (веса) нейросети;
- вход — состояние s (например, картинка с игры);
- выход — вектор из оценок $Q(s, a)$ для всех действий.

Пример

Вместо таблицы:

Состояние	Действие	Q
s_1	a_1	0.6
s_1	a_2	0.2

Теперь нейросеть получает на вход **изображение кадра** и **выдаёт значения Q для всех действий**:

$$\text{output} = [Q(s, a_1), Q(s, a_2), Q(s, a_3), \dots]$$

3. Что мы выиграли

Преимущество	Объяснение
Масштабируемость	Нейросеть обобщает знания — не нужно хранить Q для всех состояний.

Преимущество	Объяснение
Работа с изображениями и сенсорными данными	Нейросеть извлекает признаки (features) автоматически.
Эффект “внутреннего представления”	Модель учится понимать структуру среды (например, где находится игрок, цель, враги).
Интеграция с DL-инфраструктурой	Используем PyTorch, TensorFlow, GPU — ускорение в сотни раз.

4. Пример: классический Q-Learning vs Deep Q-Learning

Характеристика	Q-Learning	Deep Q-Learning (DQN)
Представление Q	Таблица	Нейросеть
Тип среды	Малые дискретные	Большие, визуальные
Память	Огромная	Аппроксимированная
Генерализация	Нет	Да
Пример	FrozenLake	Atari, CarRacing

5. Как работает Deep Q-Learning (интуиция)

Мы заменяем Q-таблицу на нейросеть и обучаем её с помощью **метода обратного распространения ошибки** (backpropagation).

Целевая функция потерь

$$L(\theta) = \mathbb{E} \left[\left(y - Q(s, a; \theta) \right)^2 \right]$$

где

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

— целевое значение (target), вычисленное с использованием **замороженной сети** θ^- .

Ключевые улучшения от DeepMind (2015)

1. **Experience Replay** — буфер памяти, где агент хранит опыт (s, a, r, s') и обучается на случайных мини-батчах. → уменьшает корреляцию данных.
2. **Target Network** — копия основной сети, обновляется реже. → стабилизирует обучение.

Результат: DeepMind обучила агента **играть в Atari 2600** без знания правил игры — только по пикселям! Это стало **вехой в истории RL (Nature, 2015)**.

6. Когда "Deep" помогает, а когда — нет

Ситуация	Подход
Простая дискретная среда (FrozenLake, GridWorld)	Классический RL
Сложные визуальные данные (Atari, Doom, Robot sensors)	Deep RL
Ограниченные данные или вычислительная мощность	Классический RL
Большие состояния и непрерывные действия	Deep RL (PPO, SAC, TD3 и др.)

7. Итог

Deep Reinforcement Learning — это симбиоз **обучения с подкреплением** и **глубоких нейросетей**, где нейросеть служит **аппроксиматором** для функций ценности, политики или обеих сразу.

Главное запомнить

Компонент	Классический RL	Deep RL
Представление состояния	Вектор/индекс	Картинка, сигнал, embedding
Функция $Q(s, a)$	Таблица	Нейросеть
Аппроксимация	Нет	Есть

Компонент	Классический RL	Deep RL
Обучение	Табличное обновление	Градиентный спуск
Применение	Простые среды	Сложные визуальные и непрерывные задачи

Основано на:

- Sutton & Barto, *Reinforcement Learning: An Introduction* (2020)
- Mnih et al., *Human-level control through deep reinforcement learning* (Nature, 2015)
- Andrea Lonza, *Алгоритмы обучения с подкреплением на Python* (2020)